

ИНТЕЛЛЕКТ ➤

# Fine-tuning, LoRa

к.ф.-м.н. Тихомиров Михаил Михайлович

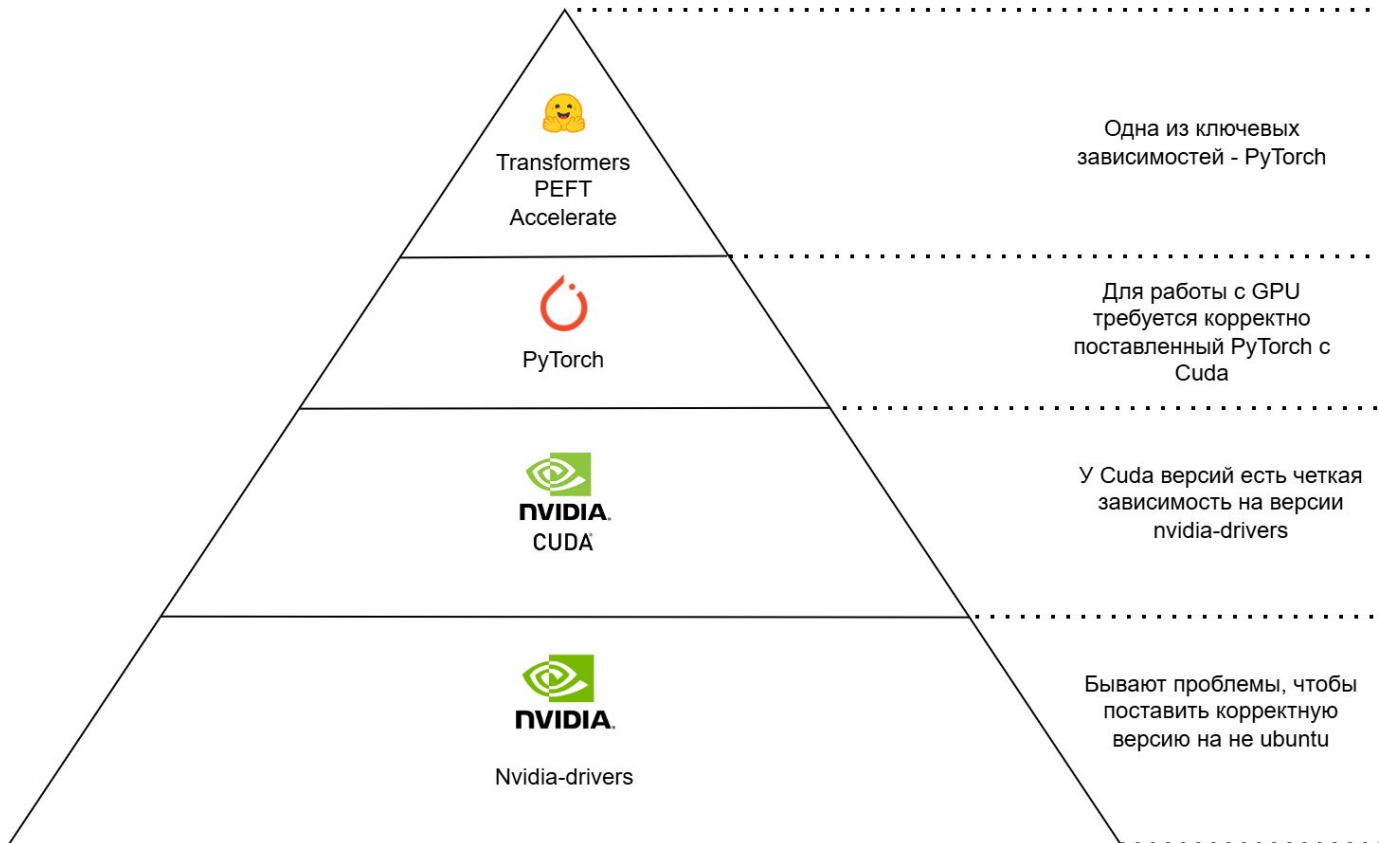
старший научный сотрудник  
НИВЦ МГУ имени М. В. Ломоносова

- Иногда zero-shot и few-shot не достаточно.
- Модель **7b** параметров при загрузке на видеокарту в **fp16** занимает около **15gb** (и это даже не во время генерации или обучения), но хочется **fine-tuning**?
  - Полный fine-tuning, используя классические методы.
  - P-tuning, Prompt-tuning, Prefix-tuning.
  - LoRa / QLoRa.

# “Пирамида” технологий LLM



Московский  
государственный  
университет  
имени М.В. Ломоносова



# Pytorch



Московский  
государственный  
университет  
имени М.В. Ломоносова

```
[1] import torch  
    torch.cuda.is_available()
```

True

PyTorch Build

Your OS

Package

Language

Compute Platform

Run this Command:

Stable (2.5.1)

Preview (Nightly)

Linux

Mac

Windows

Conda

Pip

LibTorch

Source

Python

C++ / Java

CUDA  
11.8

CUDA  
12.1

CUDA  
12.4

ROCm 6.2

CPU

```
pip3 install torch torchvision torchaudio --index-url https://download.pytorch.  
org/whl/cu118
```

Библиотека для  
вычислений на GPU

В отличие от  
драйверов может  
быть поставлена в  
docker контейнере.

| CUDA Toolkit                                                                                                         | Minimum Required Driver Version for CUDA Minor Version Compatibility* |                               |
|----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|-------------------------------|
|                                                                                                                      | Linux x86_64 Driver Version                                           | Windows x86_64 Driver Version |
| CUDA 12.x                                                                                                            | >=525.60.13                                                           | >=528.33                      |
| CUDA 11.8.x<br>CUDA 11.7.x<br>CUDA 11.6.x<br>CUDA 11.5.x<br>CUDA 11.4.x<br>CUDA 11.3.x<br>CUDA 11.2.x<br>CUDA 11.1.x | >=450.80.02                                                           | >=452.39                      |
| CUDA 11.0<br>(11.0.3)                                                                                                | >=450.36.06**                                                         | >=451.22**                    |

## CUDA Toolkit 11.8 Downloads

### Select Target Platform

Click on the green buttons that describe your target platform. Only supported platforms will be shown. By downloading and using the software, you agree to fully comply with the terms and conditions of the [CUDA EULA](#).

|                  |             |               |                 |                |          |      |       |      |        |
|------------------|-------------|---------------|-----------------|----------------|----------|------|-------|------|--------|
| Operating System | Linux       | Windows       |                 |                |          |      |       |      |        |
| Architecture     | x86_64      | ppc64le       | arm64-sbsa      | aarch64-jetson |          |      |       |      |        |
| Distribution     | CentOS      | Debian        | Fedora          | KylinOS        | OpenSUSE | RHEL | Rocky | SLES | Ubuntu |
|                  | WSL-Ubuntu  |               |                 |                |          |      |       |      |        |
| Version          | 18.04       | 20.04         | 22.04           |                |          |      |       |      |        |
| Installer Type   | deb (local) | deb (network) | runfile (local) |                |          |      |       |      |        |

**Download Installer for Linux Ubuntu 22.04 x86\_64**

The base installer is available for download below.

**> Base Installer**

Installation Instructions:

```
$ wget https://developer.download.nvidia.com/compute/cuda/11.8.0/local_installers/cuda_11.8.0_520.61.05_linux.run
$ sudo sh cuda_11.8.0_520.61.05_linux.run
```

Можно ставить и через apt-get / аналоги

```
sudo ubuntu-drivers install nvidia:535
```

Предварительно рекомендуется удалить все предыдущие версии

```
sudo apt remove --purge '^nvidia-.*'  
sudo apt remove --purge '^libnvidia-.*'
```

Но обычно проще ставить через installer без установки cuda, если она не нужна.

# Docker

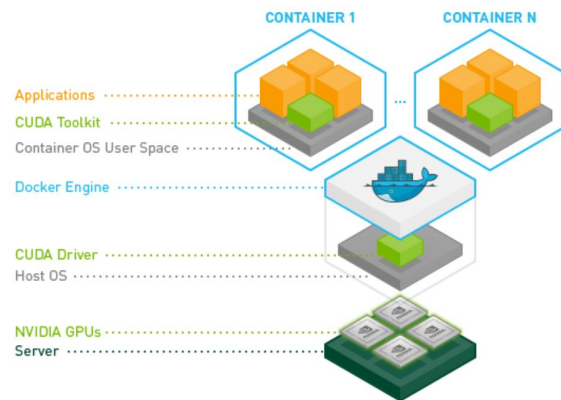
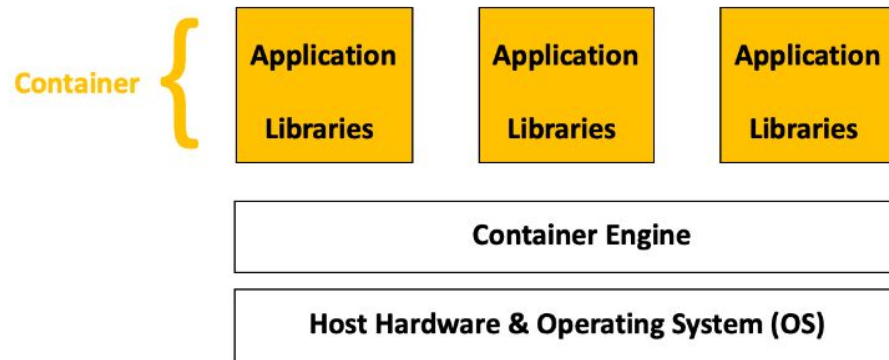


Московский  
государственный  
университет  
имени М.В. Ломоносова

Виртуализация на уровне ОС, библиотек и приложений.

Драйвера все еще идут от Host OS, например, nvidia-drivers

OS, python, CUDA, pytorch ... все можно ставить в docker контейнере





# Docker: основные понятия

- Dockerfile
  - Конфигурация нашего образа
- Docker image
  - Собранный образ, может быть экспортирован
- Docker container
  - Запущенный docker image

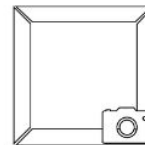
1. Dockerfile



Build



2. Docker Image



Run



3. Docker Container(s)



# Docker image: ngc



Для обучения в **multi-node** настоятельно рекомендуется брать **ngc** контейнер за базу (и следить, чтобы не переставлялся **pytorch**)!

Иначе:



docs.nvidia.com/deeplearning/frameworks/pytorch-release-notes/rel-24-08.html

NVIDIA DOCS HUB

Search all documentation

## NVIDIA Optimized Frameworks

### Topics

1. PyTorch Overview
2. Pulling A Container
3. Running PyTorch
4. PyTorch Release 24.10
5. PyTorch Release 24.09
6. PyTorch Release 24.08
7. PyTorch Release 24.07
8. PyTorch Release 24.06
9. PyTorch Release 24.05
10. PyTorch Release 24.04
11. PyTorch Release 24.03
12. PyTorch Release 24.02
13. PyTorch Release 24.01
14. PyTorch Release 23.12
15. PyTorch Release 23.11
16. PyTorch Release 23.10
17. PyTorch Release 23.09
18. PyTorch Release 23.08
19. PyTorch Release 23.07
20. PyTorch Release 23.06
21. PyTorch Release 23.05
22. PyTorch Release 23.04
23. PyTorch Release 23.03

NVIDIA Docs Hub > NVIDIA Optimized Frameworks > NVIDIA Optimized Frameworks > PyTorch

[Download PDF](#)

## PyTorch Release 24.08

The NVIDIA container image for PyTorch, release 24.08, is available on [NGC](#).

### Contents of the PyTorch container

This container image contains the complete source of the version of PyTorch in `/opt/pytorch` image. The container also includes the following:

- Ubuntu 22.04 including [Python 3.10](#)
- NVIDIA CUDA 12.6
- NVIDIA cuBLAS 12.6.0.22
- NVIDIA cuDNN 9.3.0.75
- NVIDIA NCCL 2.22.3
- NVIDIA RAPIDS™ 24.06
  - rdma-core 39.0
  - NVIDIA HPC-X 2.19
- OpenMPI 4.1.7
- GDRCopy 2.3
- TensorBoard 2.16.2
- [Night Compute 2024.3.0.15](#)
- [Night Systems 2024.4.2.133](#)
- NVIDIA TensorRT™ 10.3.0.26
- Torch-TensorRT 2.5.0a0
- NVIDIA DALI 1.40
- [mImageCodec 0.2.0.7](#)
- MAGMA 2.6.2
- [JupyterLab 4.2.4](#) including [Jupyter-TensorBoard](#)
- [TransformerEngine 1.9](#)
- TensorRT Model Optimizer 0.15.0

## Key Features and Enhancements

This PyTorch release includes the following key features and enhancements.

- [PyTorch](#) container image version 24.08 is based on [2.5.0a0+872d972e41](#).

# Docker container



- Контейнер - запущенный image
- Все внутреннее состояние контейнера независимо от остальных контейнеров (не считая связанных volume):
  - Запустили контейнер, сделали что-то в нем, перезапустили - контейнер снова в начальном состоянии
- Чтобы пробрасывать gpu должен стоять `nvidia-container-toolkit / nvidia-docker2`

# Docker container



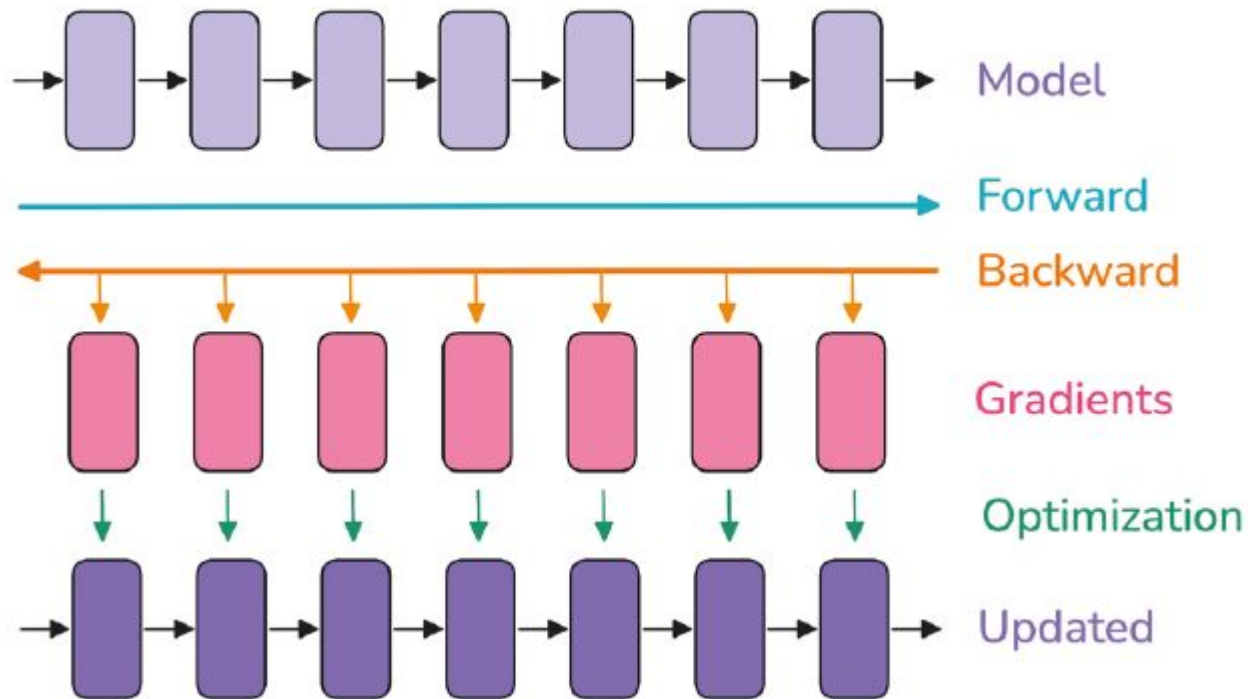
```
docker run -v /home/ubuntu:/workdir -p 8000:8888 -it --gpus '"device=0,1"' --rm --name jupyter01 ngc_cuda_pytorch_vllm_11_10_24_v7 bash
```

- -v /host\_path:/container\_path
- -p host\_port:container\_port
  - При таком варианте открывает его наружу!
- --gpus all “пробросит” в контейнер все gpus машины
- --name jupyter01 - имя контейнера для удобства, --rm - после завершения работы контейнера, он будет удален (не image!)
- ngc\_cuda... название image
- bash - команда на запуске, -it - интерактивный режим
- По умолчанию требуются sudo права, если не настроено иначе

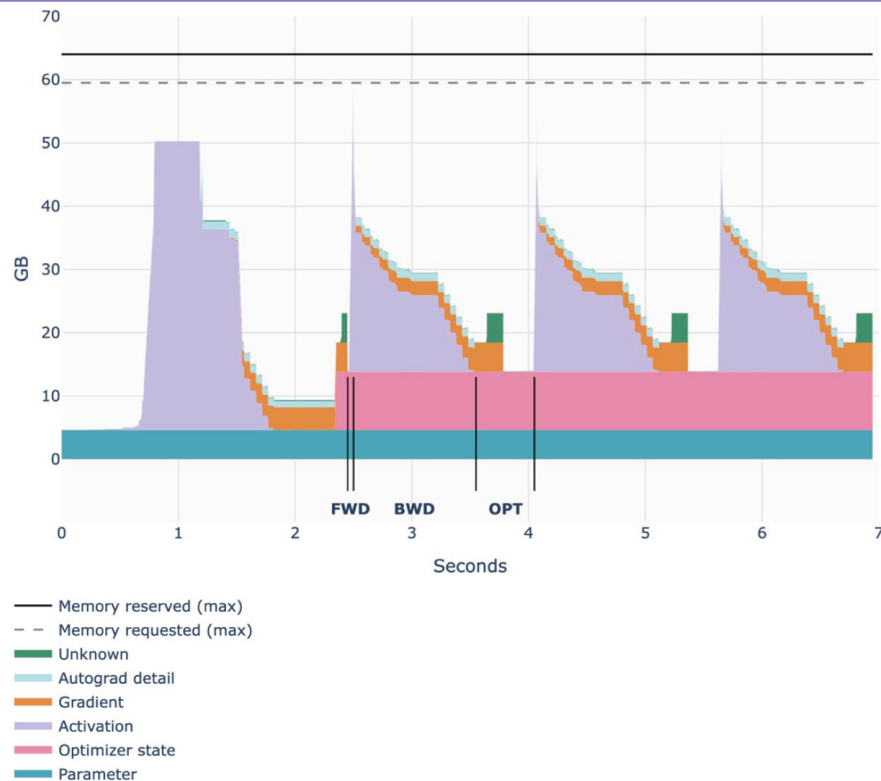
# Процесс обучения



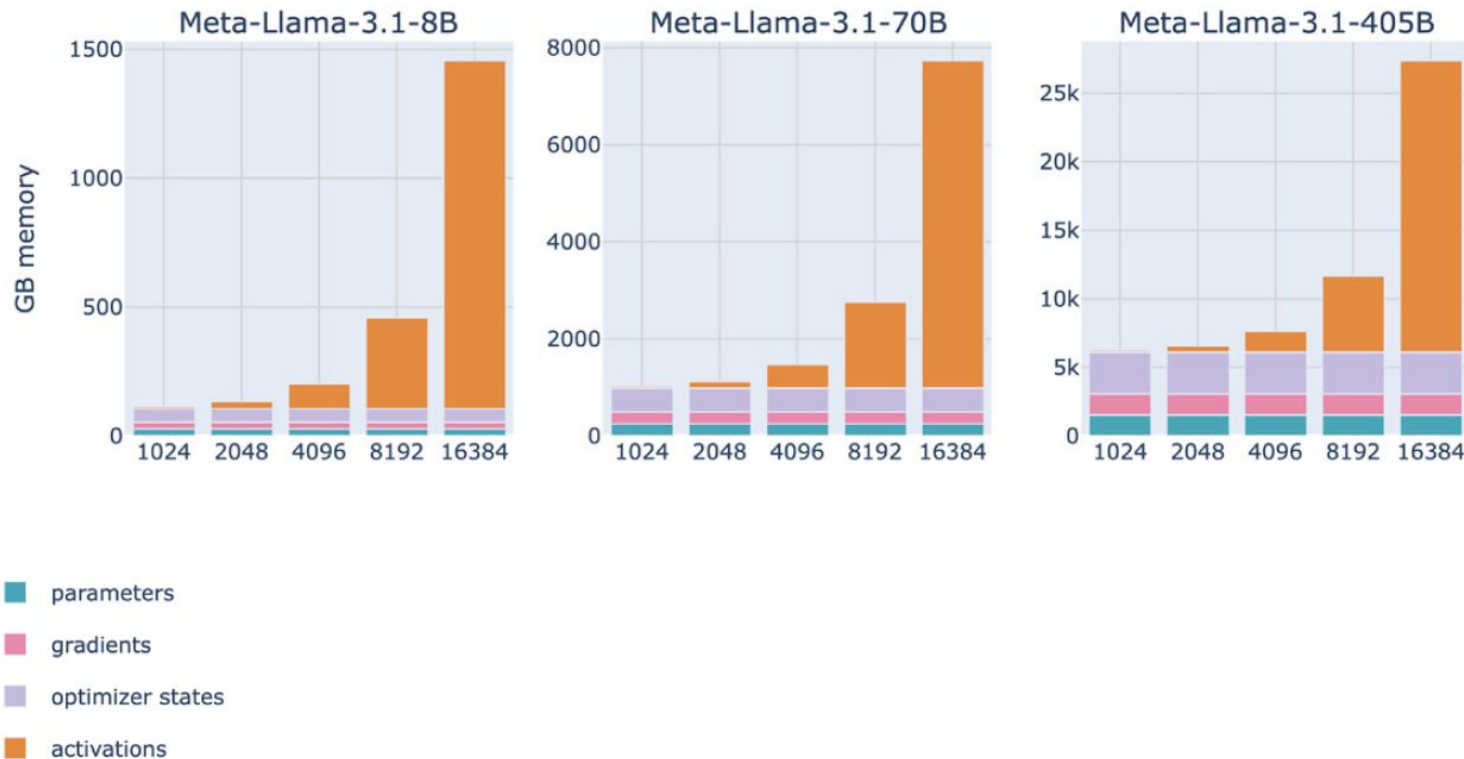
Московский  
государственный  
университет  
имени М.В. Ломоносова



# На что уходит память



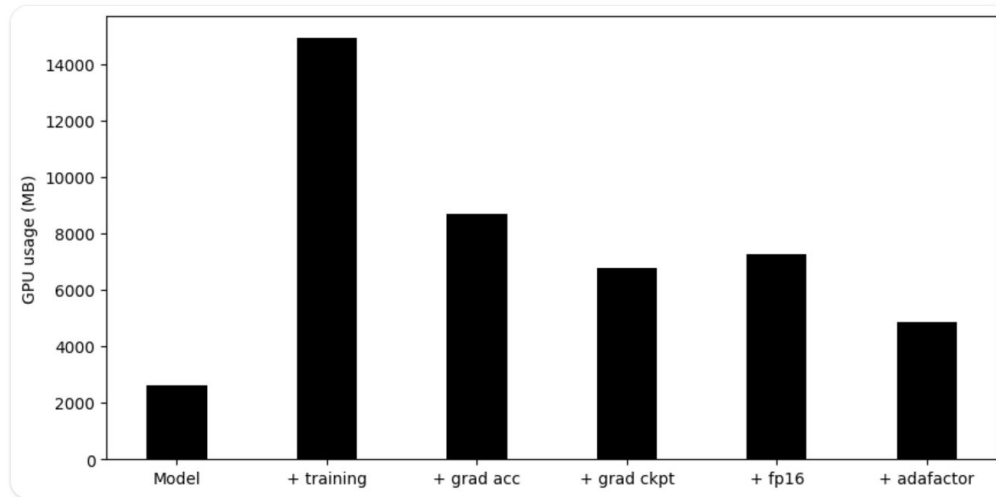
# Активации





# Fine-tuning: способы оптимизации

- Gradient Accumulation.
- Gradient Checkpointing.
  - Активации с forward pass можно не сохранять, а пересчитывать во время backward.
- Mixed-precision training (fp16 / bf16).
  - Выигрыш обычно сильно больше.
- Adafactor как альтернатива AdamW или paged\_adamw\_8bit.
  - Могут быть проблемы со стабильностью





# Fine-tuning



- **Multi-gpu** система, V100 / **A100**,
- Использовать **DDP** (distributed data parallel), **gradient checkpointing**, маленький **batch size** + **accumulation gradients**,
- Пакет **DeepSpeed** от Microsoft.

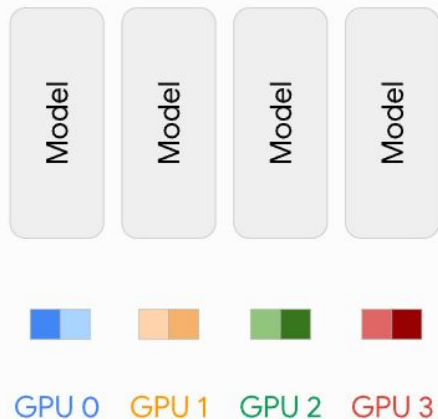
# Виды параллелизма



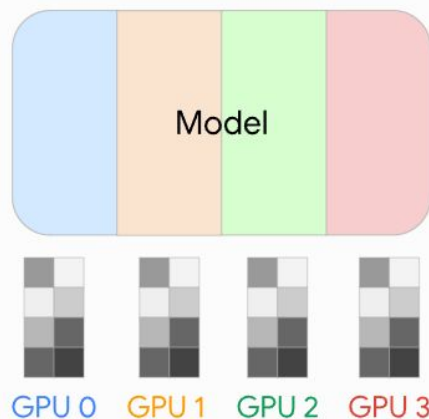
## Single GPU



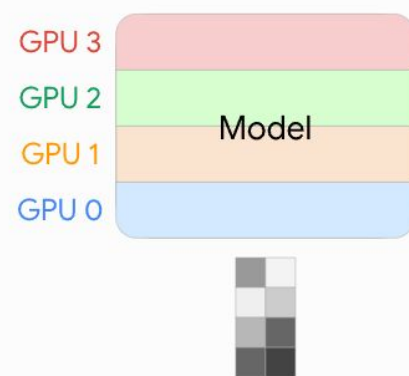
## Data Parallelism



## Tensor Parallelism



## Pipeline Parallelism

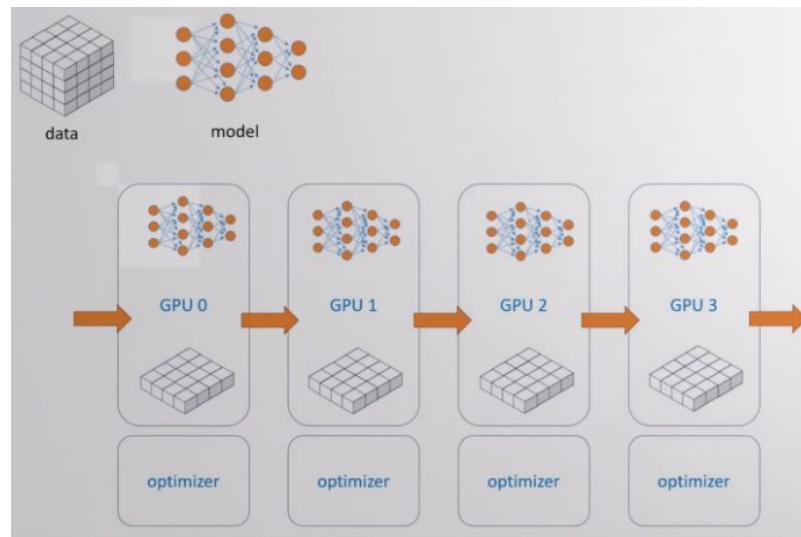


# Data Parallelism



**Data Parallelism (DP)**- На каждой GPU копия модели, но данные из батча разделяются по разным GPU. Каждая GPU накапливает свои градиенты за шаг оптимизации, потом шаг синхронизации.

- Предназначен для 1 ноды
- Используется многопоточность, а не многопроцессорность
- Шаг оптимизации происходит на GPU 0, затем синхронизация на остальных



# Distributed Data Parallelism (DDP)

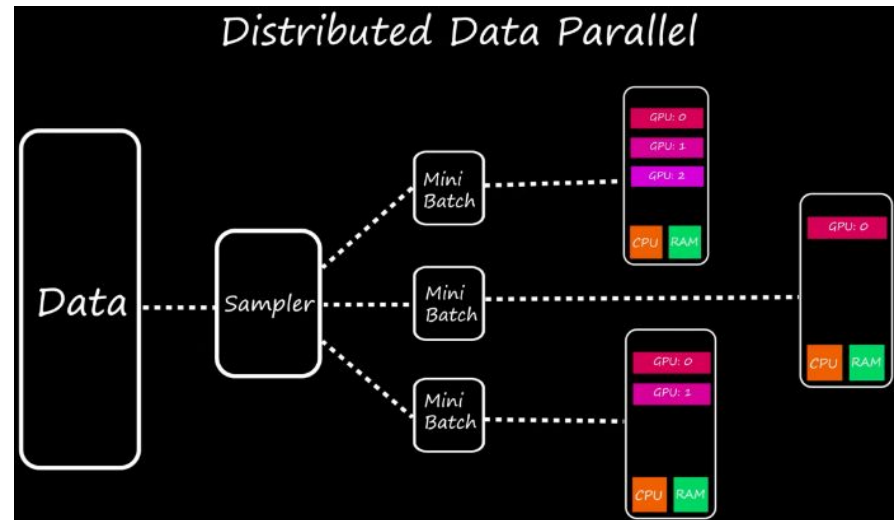


Логика полностью повторяет DP, но:

- Можно multi-node
- Используется multiprocessing
- Обычно используется DDP

Когда можно и нужно использовать DDP:

- Модель влезает на GPU полностью + хватает места для процесса обучения

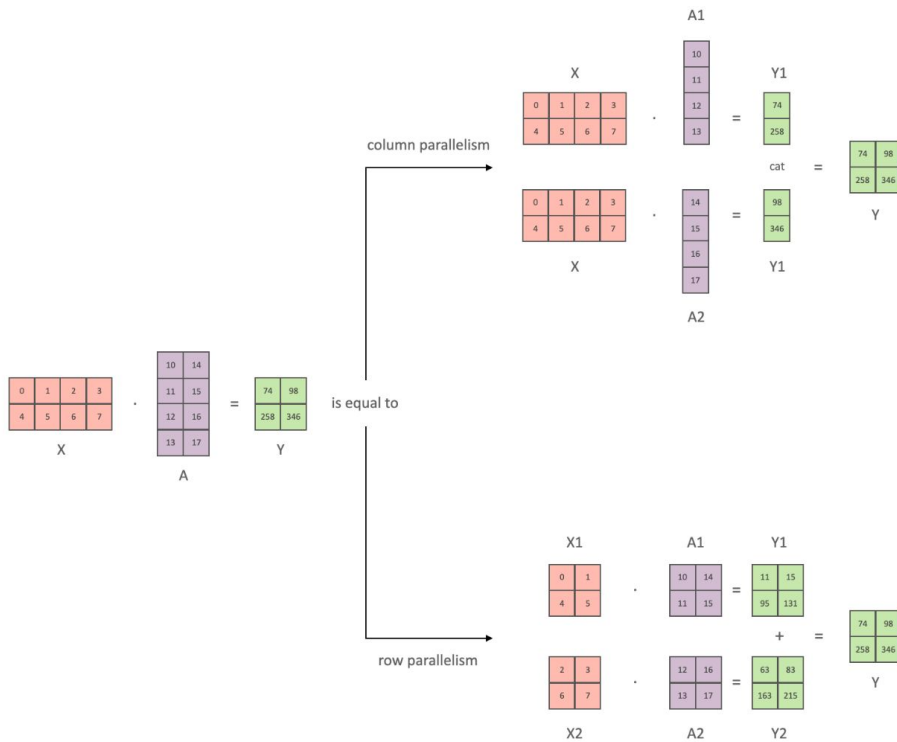


# Tensor Parallelism



**Tensor Parallelism** - Модель “разрезается” на разные GPU через все слои. Каждая GPU считает свою часть тензорных операций, затем происходит агрегация.

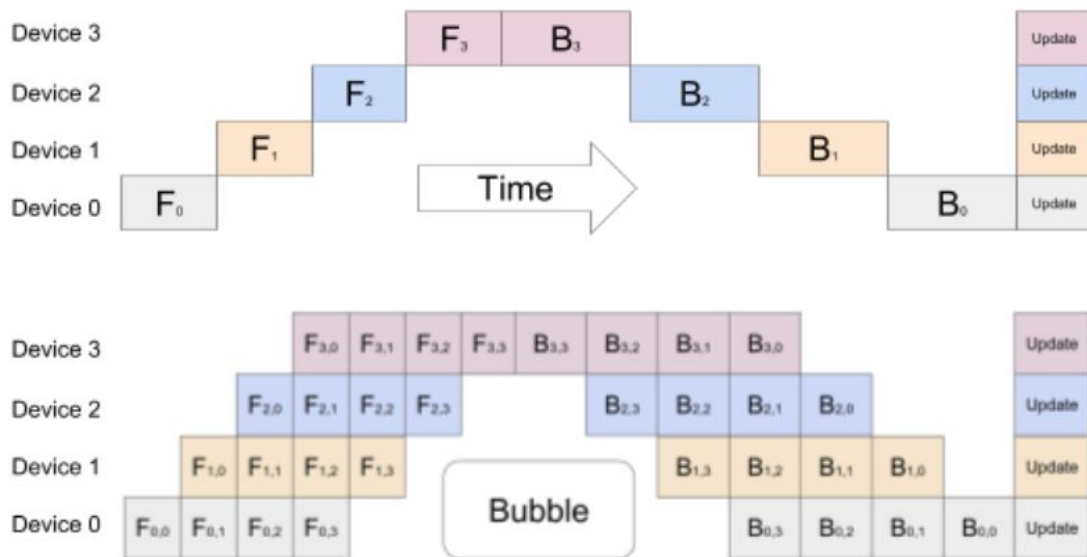
Используется для multigpu inference



# Pipeline Parallelism



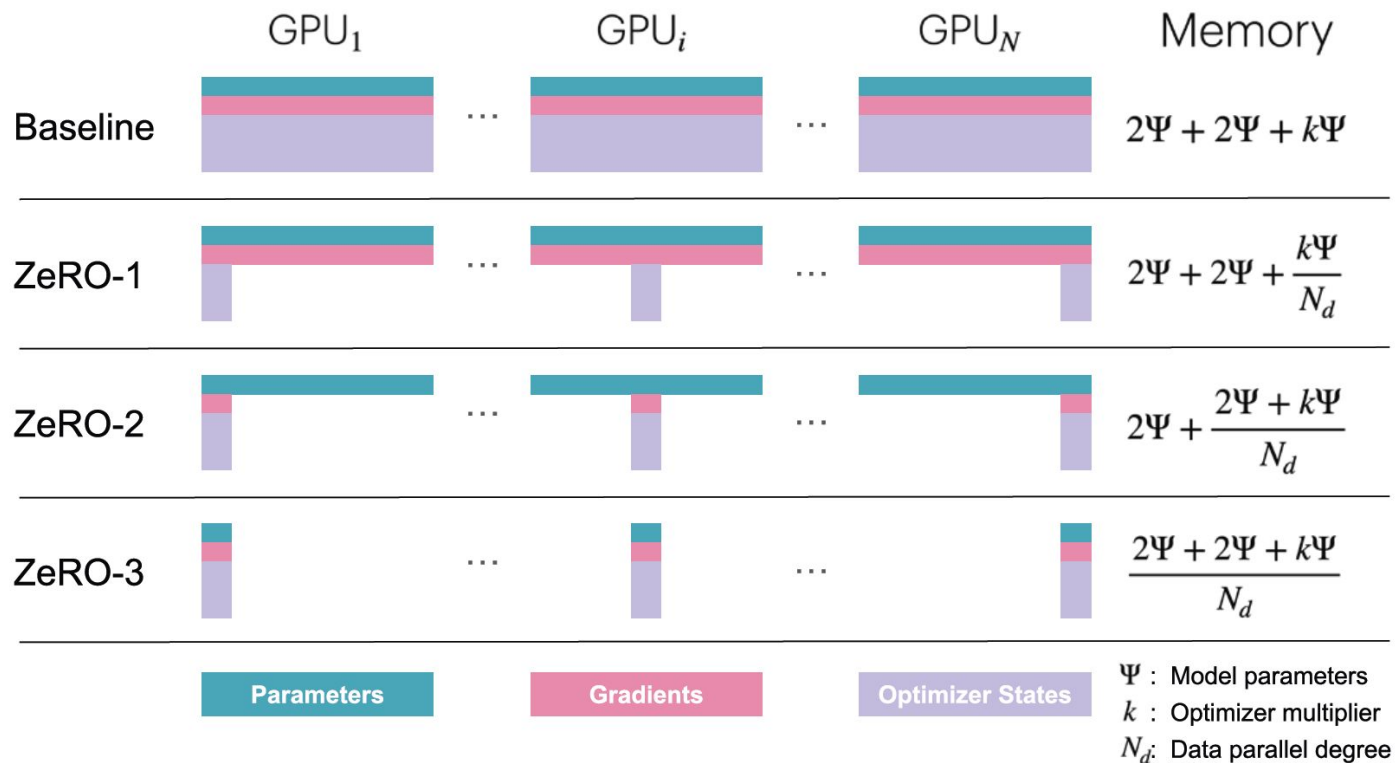
**Pipeline Parallelism** - Модель разрезается по слоям на разные GPU. Конвейерная обработка.



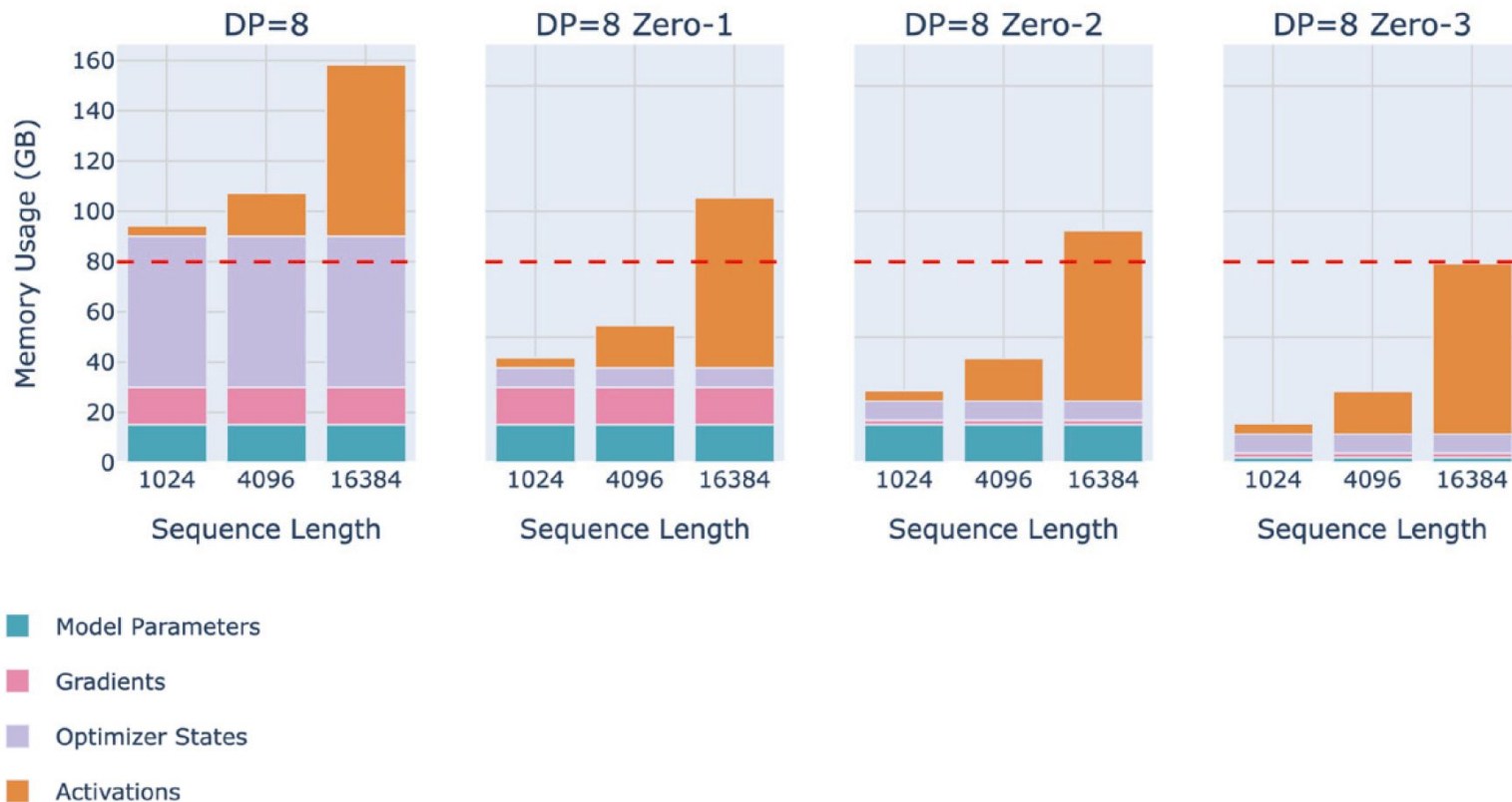
# DeepSpeed



Московский  
государственный  
университет  
имени М.В. Ломоносова



# Deepspeed

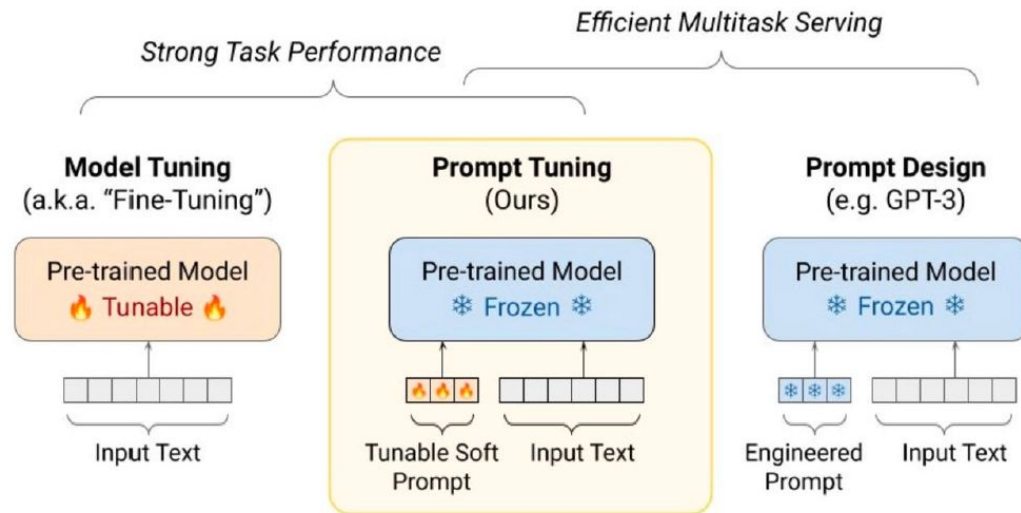




# Prompt-tuning



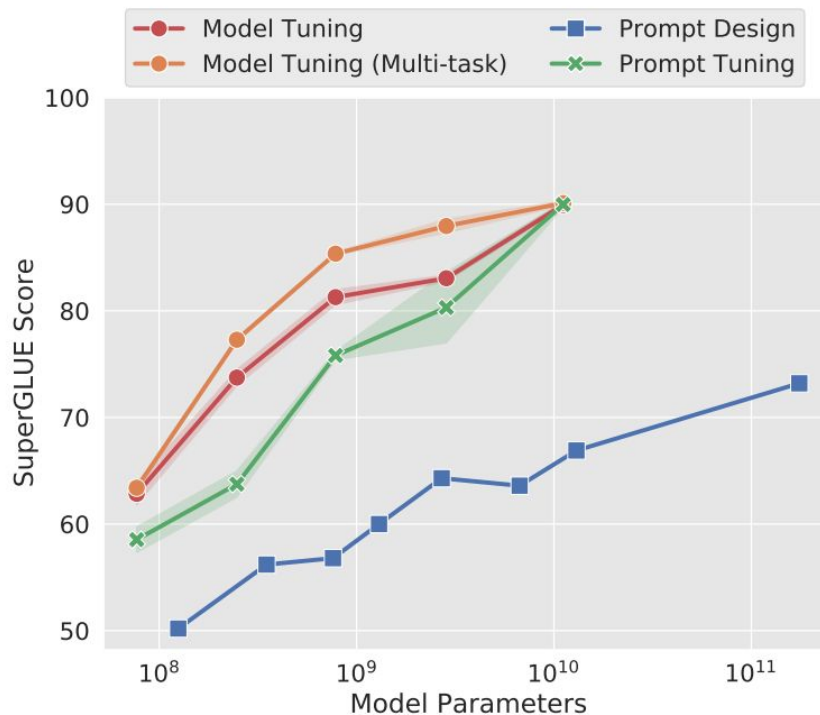
- Альтернатива дообучению,
- Вместо подбора слов (токенов) промпта, **подбираются входные эмбединги** для нескольких токенов входного текста.
- Напрямую обучаются “soft prompt” токены.



# Prompt-tuning: результаты

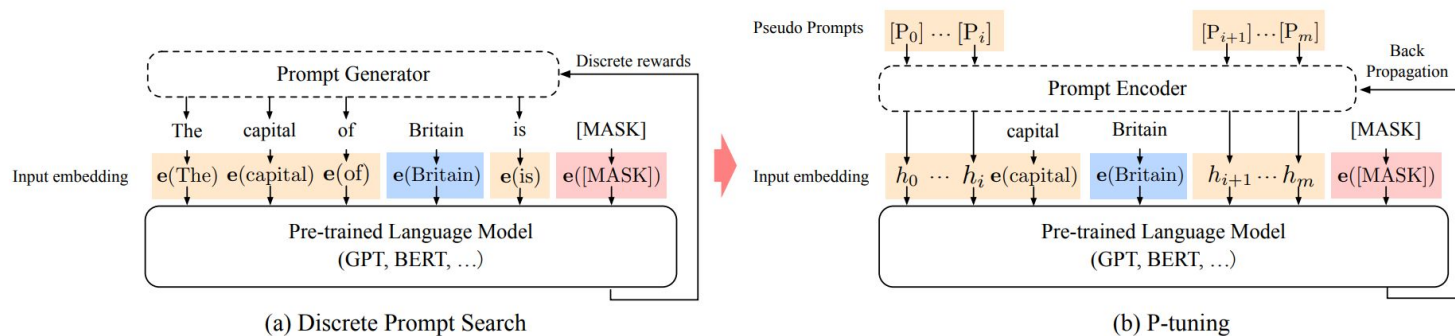


| Dataset    | Domain | Model                 | Prompt                | $\Delta$ |
|------------|--------|-----------------------|-----------------------|----------|
| SQuAD      | Wiki   | $94.9 \pm 0.2$        | $94.8 \pm 0.1$        | -0.1     |
| TextbookQA | Book   | $54.3 \pm 3.7$        | <b>66.8</b> $\pm 2.9$ | +12.5    |
| BioASQ     | Bio    | $77.9 \pm 0.4$        | <b>79.1</b> $\pm 0.3$ | +1.2     |
| RACE       | Exam   | $59.8 \pm 0.6$        | <b>60.7</b> $\pm 0.5$ | +0.9     |
| RE         | Wiki   | $88.4 \pm 0.1$        | <b>88.8</b> $\pm 0.2$ | +0.4     |
| DuoRC      | Movie  | <b>68.9</b> $\pm 0.7$ | $67.7 \pm 1.1$        | -1.2     |
| DROP       | Wiki   | <b>68.9</b> $\pm 1.7$ | $67.1 \pm 1.9$        | -1.8     |



- Еще одна альтернатива дообучению,
- Вместо подбора слов (токенов) промпта, **подбираются входные эмбединги** для нескольких токенов входного текста через специальный **prompt encoder**.

## GPT Understands, Too



Промт энкодер нужно обучать!

# P-tuning: результаты



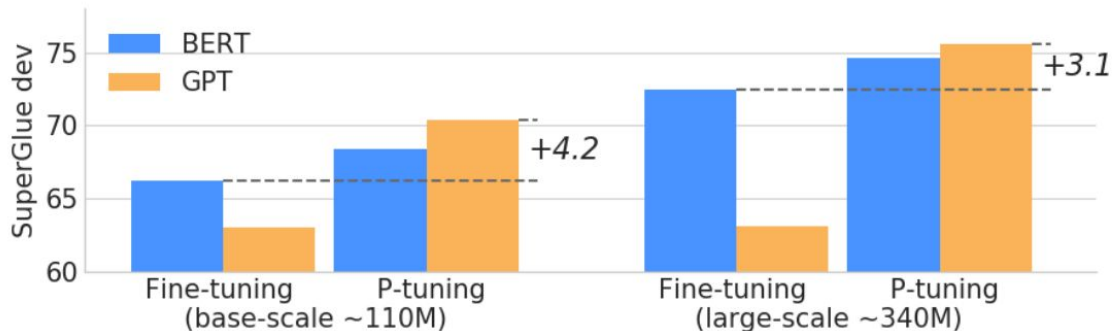
- Датасет LAMA-29
  - “пробинг” знаний моделей,

(Dante, born-in, Florence) ->  
“Dante was born in [MASK].

| Model                           | MP   | FT   | MP+FT | P-tuning            |
|---------------------------------|------|------|-------|---------------------|
| BERT-base (109M)                | 31.7 | 51.6 | 52.1  | 52.3 (+20.6)        |
| -AutoPrompt (Shin et al., 2020) | -    | -    | -     | 45.2                |
| BERT-large (335M)               | 33.5 | 54.0 | 55.0  | 54.6 (+21.1)        |
| RoBERTa-base (125M)             | 18.4 | 49.2 | 50.0  | 49.3 (+30.9)        |
| -AutoPrompt (Shin et al., 2020) | -    | -    | -     | 40.0                |
| RoBERTa-large (355M)            | 22.1 | 52.3 | 52.4  | 53.5 (+31.4)        |
| GPT2-medium (345M)              | 20.3 | 41.9 | 38.2  | 46.5 (+26.2)        |
| GPT2-xl (1.5B)                  | 22.8 | 44.9 | 46.5  | 54.4 (+31.6)        |
| MegatronLM (11B)                | 23.1 | OOM* | OOM*  | <b>64.2</b> (+41.1) |

\* MegatronLM (11B) is too large for effective fine-tuning.

- Бенчмарк SuperGlue



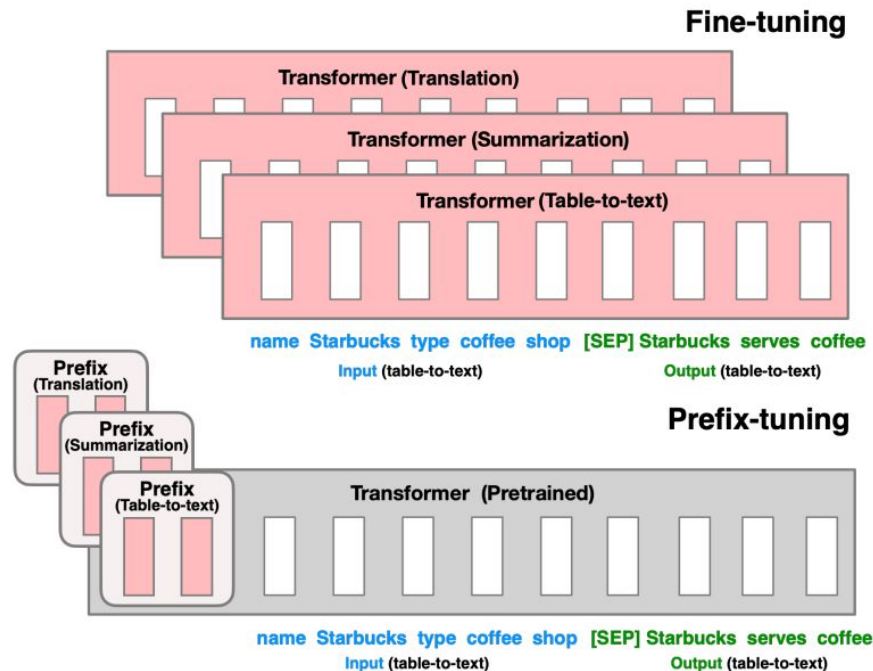
# Prefix-tuning



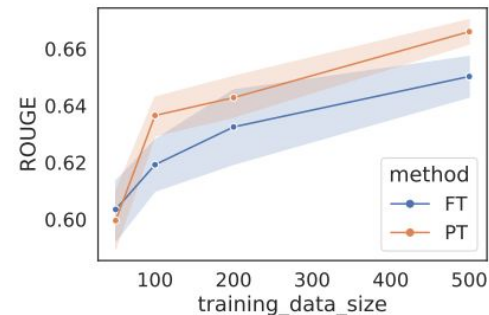
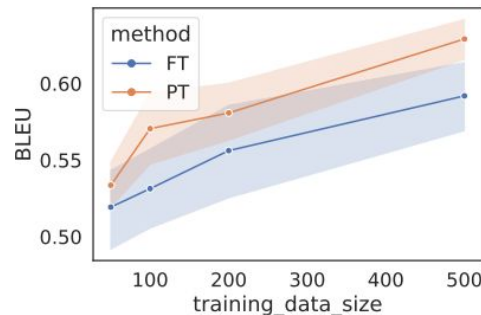
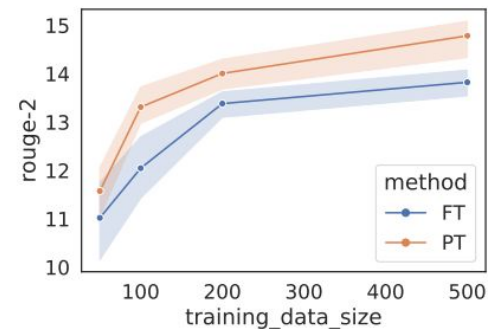
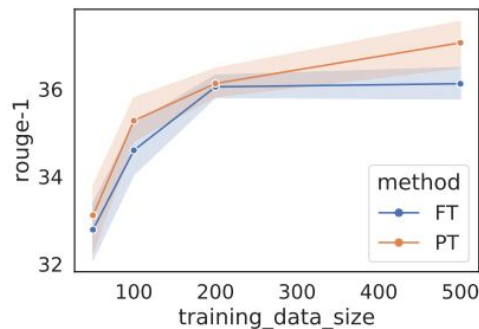
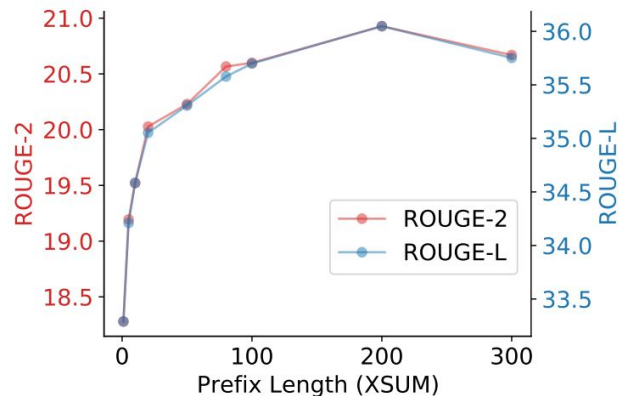
- Продолжение предыдущих идей.
- Обучаются специальные эмбединги префикса, которые добавляются на каждом трансформер блоке.
- Эмбединги обучаются не напрямую, а через полносвязный слой.

$$h_i = \begin{cases} P_\theta[i, :], & \text{if } i \in P_{\text{idx}}, \\ \text{LM}_\phi(z_i, h_{<i}), & \text{otherwise.} \end{cases}$$

$P_\theta[i, :] = \text{MLP}_\theta(P'_\theta[i, :])$  by a smaller matrix ( $P'_\theta$ )



# Prefix-tuning: результаты



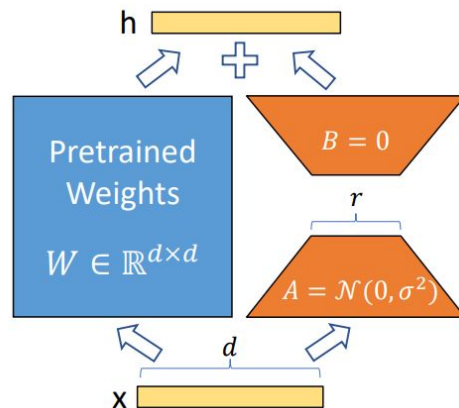
|                               | R-1 ↑ | R-2 ↑ | R-L ↑ |
|-------------------------------|-------|-------|-------|
| FINE-TUNE(Lewis et al., 2020) | 45.14 | 22.27 | 37.25 |
| PREFIX(2%)                    | 43.80 | 20.93 | 36.05 |
| PREFIX(0.1%)                  | 42.92 | 20.03 | 35.05 |

- То, благодаря чему мы имеем saiga модели,
  - Первые версии **Saiga-7b** и **13b** обучались на всего одной **RTX 3090**,
- **Позволяет обучать** всю сеть, но при этом **уменьшая** количество **обучаемых** параметров **в 10,000 раз** (для GPT-3),
  - И требования к памяти GPU в  $N$  раз,
- Основная идея в том, чтобы не обучать все параметры модели, а только некоторую “добавку”, причем в **low-rank**,
- После слияния с моделью имеем новую модель **без дополнительных затрат** на работу.

# LoRa: основная идея



$$W_0 + \Delta W = W_0 + BA, \text{ where } B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k},$$



We then scale  $\Delta W x$  by  $\frac{\alpha}{r}$

- Обучаются матрицы  $B$  и  $A$ ,
- Соответственно для каждого  $Q, K, V$  в attention можно применить подобный “трюк”.



# LoRa: alpha



$$W_0 + \Delta W = W_0 + BA, \text{ where } B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k},$$



$$h = W_0 x + \Delta W x = W_0 x + BAx$$

We then scale  $\Delta W x$  by  $\frac{\alpha}{r}$

- Не сразу очевидный результат введения альфы - возможность изменить ее перед слиянием с моделью, в случае, если есть переобучение.
  - Добавку BA можно интерпретировать как накопленный градиент за обучение, меняя альфу мы меняем “learning rate” (хотя “шаг” один).

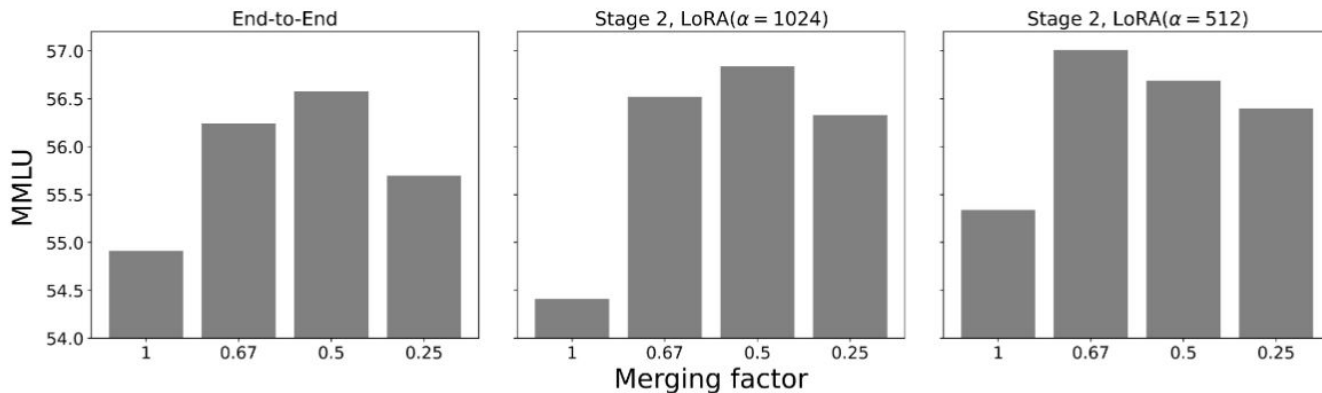


Figure 5. The effect of LoRA adapter merging factor on model performance

# LoRa: результаты



| Model&Method                  | # Trainable<br>Parameters | WikiSQL     | MNLI-m      | SAMSum                |
|-------------------------------|---------------------------|-------------|-------------|-----------------------|
|                               |                           | Acc. (%)    | Acc. (%)    | R1/R2/RL              |
| GPT-3 (FT)                    | 175,255.8M                | <b>73.8</b> | 89.5        | 52.0/28.0/44.5        |
| GPT-3 (BitFit)                | 14.2M                     | 71.3        | 91.0        | 51.3/27.4/43.5        |
| GPT-3 (PreEmbed)              | 3.2M                      | 63.1        | 88.6        | 48.3/24.2/40.5        |
| GPT-3 (PreLayer)              | 20.2M                     | 70.1        | 89.5        | 50.8/27.3/43.5        |
| GPT-3 (Adapter <sup>H</sup> ) | 7.1M                      | 71.9        | 89.8        | 53.0/28.9/44.8        |
| GPT-3 (Adapter <sup>H</sup> ) | 40.1M                     | 73.2        | <b>91.5</b> | 53.2/29.0/45.1        |
| GPT-3 (LoRA)                  | 4.7M                      | 73.4        | <b>91.7</b> | <b>53.8/29.8/45.9</b> |
| GPT-3 (LoRA)                  | 37.7M                     | <b>74.0</b> | <b>91.6</b> | 53.4/29.2/45.1        |

# LoRa: какие матрицы дообучать



|                          | # of Trainable Parameters = 18M |            |            |            |                 |                 |                           |
|--------------------------|---------------------------------|------------|------------|------------|-----------------|-----------------|---------------------------|
| Weight Type<br>Rank $r$  | $W_q$<br>8                      | $W_k$<br>8 | $W_v$<br>8 | $W_o$<br>8 | $W_q, W_k$<br>4 | $W_q, W_v$<br>4 | $W_q, W_k, W_v, W_o$<br>2 |
| WikiSQL ( $\pm 0.5\%$ )  | 70.4                            | 70.0       | 73.0       | 73.2       | 71.4            | <b>73.7</b>     | <b>73.7</b>               |
| MultiNLI ( $\pm 0.1\%$ ) | 91.0                            | 90.8       | 91.0       | 91.3       | 91.3            | 91.3            | <b>91.7</b>               |

- Известный фреймворк для более быстрого instruction-tuning
- Обычный сценарий: LoRa + int4 + unsloth
- Плюсы: быстрый, минимальные изменения в коде
- Минусы: только single-gpu

| 1 A100 40GB     | Dataset    | 🤗 Hugging Face | 🚀 + Flash Attention 2 | 👉 Unsloth | 👉 VRAM reduction |
|-----------------|------------|----------------|-----------------------|-----------|------------------|
| Code Llama 34b  | Slim Orca  | 1x             | 1.01x                 | 1.94x     | -22.7%           |
| Llama-2 7b      | Slim Orca  | 1x             | 0.96x                 | 1.87x     | -39.3%           |
| Mistral 7b      | Slim Orca  | 1x             | 1.17x                 | 1.88x     | -65.9%           |
| Tiny Llama 1.1b | Alpaca     | 1x             | 1.55x                 | 2.74x     | -57.8%           |
| DPO with Zephyr | Ultra Chat | 1x             | 1.24x                 | 1.88x     | -11.6%           |

| Free Colab T4   | Dataset    | 🤗 Hugging Face | 🚀 + Pytorch 2.1.1 | 👉 Unsloth | 👉 VRAM reduction |
|-----------------|------------|----------------|-------------------|-----------|------------------|
| Llama-2 7b      | OASST      | 1x             | 1.19x             | 1.95x     | -43.3%           |
| Mistral 7b      | Alpaca     | 1x             | 1.07x             | 1.56x     | -13.7%           |
| Tiny Llama 1.1b | Alpaca     | 1x             | 2.06x             | 3.87x     | -73.8%           |
| DPO with Zephyr | Ultra Chat | 1x             | 1.09x             | 1.55x     | -18.6%           |

Важный аспект при подготовке данных для обучения.

- Алгоритмы требуют на вход batch размером (bs, seq\_len), элементами которого являются токены.
- Разные последовательности имеют разную длину в токенах, но надо собрать тензор
- Как решение - добавление спец. токена pad
  - Маска внимания при этом модифицируется
  - Паддить можно как слева, так и справа.
    - При instruction-tuning обычно слева, так как batched inference возможен только с left padding
    - При pretraining pad вообще не используется в большинстве случаев
- Часто специальные коллаторы делают паддинг сами (DataCollatorForTokenClassification)